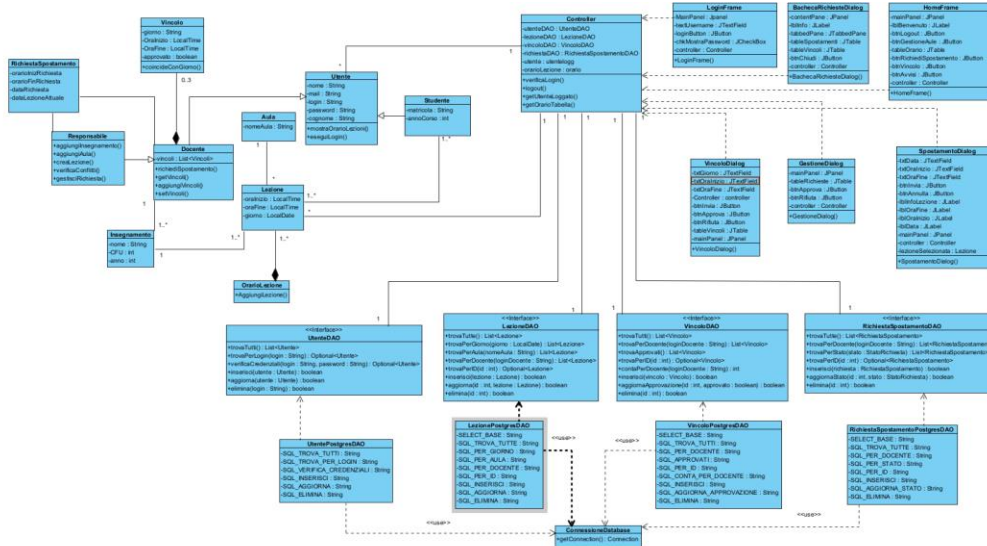


Studenti: Guida Simone (DE1000069), Iadaresta Armando (DE1000426), Maietta Francesco (DE1000431).

link repository github: <https://github.com/OOP2026/Gruppo26>



Documentazione Tecnica dello Strato di Persistenza (DAO Pattern)

1. Architettura Generale e Scelte Progettuali

Lo strato di persistenza dell'applicazione è stato progettato seguendo il Pattern architetturale DAO (Data Access Object). L'obiettivo primario di questa scelta è garantire il totale disaccoppiamento tra la logica di business dell'applicazione (rappresentata dal Controller) e la sorgente dati specifica (il DBMS relazionale PostgreSQL).

L'architettura si articola su due livelli logici: Livello di Astrazione (Package dao): Costituito da interfacce pure (UtenteDAO, LezioneDAO, VincoloDAO, RichiestaSpostamentoDAO) che definiscono il contratto formale delle operazioni CRUD (Create, Read, Update, Delete) e delle query di dominio, astruendo completamente il resto del sistema dai dettagli di persistenza. Livello di Implementazione Concrete (Package implementazioneDao): Costituito dalle classi che implementano le suddette interfacce utilizzando la tecnologia JDBC per interfacciarsi con PostgreSQL (UtentePostgresDAO, LezionePostgresDAO, ecc.).

2. Gestione della Connessione al Database

La gestione delle connessioni JDBC è centralizzata all'interno della classe `ConnessioneDatabase` (posizionata nel package `database_connection`).

La classe è stata progettata come una Utility Class non istanziabile. Presenta un costruttore esplicitamente marcato come `private` per impedire l'allocazione di istanze in memoria.

Metodo Centrico: Il metodo pubblico statico `getConnection()` : `Connection` incapsula la logica di caricamento dinamico del driver PostgreSQL (`Class.forName("org.postgresql.Driver")`) e restituisce un'istanza attiva di `java.sql.Connection`. Il metodo propaga le eventuali eccezioni tramite la clausola `throws SQLException`, delegando la gestione degli errori e del rollback transazionale alle classi DAO chiamanti.

3. Analisi di Dettaglio dei Componenti DAO

3.1 Gestione Utenti: `UtenteDAO` & `UtentePostgresDAO`

Questo modulo gestisce la persistenza degli attori del sistema, implementando una strategia di mappatura per l'ereditarietà (Single Table Strategy) a livello di database sulla tabella utenti.

Logica di Mappatura Polimorfica: Il metodo privato `mappaRiga(rs : ResultSet)` : `Utente` analizza la colonna discriminante `ruolo` (stringa). In base al valore assunto (`STUDENTE`, `DOCENTE`, `RESPONSABILE`), istanzia polimorficamente l'oggetto corretto del modello (`Studente`, `Docente`, o `Responsabile`), popolando i campi specifici come `matricola` o `anno_corso` solo se pertinenti.

Operazioni Chiave:

`verificaCredenziali(login, password)`: Consente la fase di autenticazione eseguendo una query mirata filtrata sui parametri di login e password cifrata.

`trovaPerLogin(login)`: Restituisce un oggetto avvolto in un contenitore monadico `Optional<Utente>` per gestire in modo sicuro l'eventuale assenza dell'utente nel database senza ricorrere al ritorno di valori null.

3.2 Pianificazione Orari: `LezioneDAO` & `LezionePostgresDAO`

Il modulo `Lezione` si occupa dell'orario didattico. Data la natura relazionale del DB, una singola lezione richiede l'aggregazione di più entità collegate.

Ricostruzione del Grafo di Oggetti: La costante `SELECT_BASE` implementa una complessa query SQL basata su clausole `JOIN` che collegano le tabelle lezioni, insegnamenti e utenti. Il metodo privato `mappaRiga` estrae da un'unica riga di `ResultSet` sia i dati della lezione (ora inizio, fine, aula) sia l'oggetto

Insegnamento ad essa associato, includendo a sua volta l'oggetto Docente che lo insegna.

Metodi di Filtraggio Specifici: Per supportare le viste della GUI, l'implementazione fornisce metodi ottimizzati per criteri di ricerca multipli: `trovaPerGiorno(LocalDate)`, `trovaPerAula(String)`, e `trovaPerDocente(String)`.

3.3 Gestione dei Vincoli Docente: VincoloDAO & VincoloPostgresDAO

I docenti esprimono preferenze o indisponibilità temporali che devono essere persistite ed eventualmente approvate dal Responsabile.

Integrità del Flusso: Quando un vincolo viene inserito (`inserisci(vincolo)`), il sistema imposta di default lo stato di approvazione a `false`. Il metodo `aggiornaApprovazione(id, approvato)` permette al Responsabile di modificare lo stato logico sul database.

Metodo di Controllo: Il metodo `contaPerDocente(loginDocente)` esegue una query aggregata (`SELECT COUNT(*)`) per verificare istantaneamente quanti vincoli ha sottomesso un determinato docente, supportando la logica di business che limita il numero massimo di vincoli inseribili.

3.4 Flusso Modifiche: RichiestaSpostamentoDAO & RichiestaSpostamentoPostgresDAO

Modella il ciclo di vita delle richieste effettuate dai docenti per spostare una determinata lezione.

Mappatura degli Stati Enumerati: Il database memorizza lo stato della richiesta come stringa, mentre il modello Java utilizza un tipo enumerato (`RichiestaSpostamento.StatoRichiesta`). La classe concreta effettua la conversione bidirezionale mediante i metodi `stato.name()` in scrittura e `RichiestaSpostamento.StatoRichiesta.valueOf()` in lettura.

Metodi di Workflow: Il metodo `aggiornaStato(id, stato)` consente la transizione di stato della richiesta (es. da `IN_ATTESA` ad `APPROVATA` o `RIFIUTATA`).